

Lab 7: Database Connectivity

ลิงก์ Git Repository: <https://github.com/thirawat-spec/lab7-673380039-7-sec2.git>

ส่วนที่ 1: Software Design & Principles Explanation

1.1 สถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture) และ GRASP Patterns

ระบบนี้ถูกออกแบบโดยใช้สถาปัตยกรรมแบบ Layered Architecture (MVC) ซึ่งแบ่งออกเป็นชั้นหลักๆ เพื่อความเป็นระเบียบและง่ายต่อการดูแลรักษา

- **Controller Layer (GameController):** ทำหน้าที่ตาม Controller Pattern (GRASP) เป็นจุดรับ HTTP Request จากผู้ใช้งาน (Browser) และประสานงานกับ Service
- **Service Layer (GameService):** ทำหน้าที่เป็นตัวกลาง (Indirection) ตามหลักการของ GRASP จัดการ Business Logic ทั้งหมดโดยไม่ยุ่งเกี่ยวกับการแสดงผล
- **Repository Layer (GameRepository):** ทำหน้าที่ตามหลัก Information Expert (GRASP) เป็นผู้เชี่ยวชาญเพียงผู้เดียวที่รู้วิธีดึงและบันทึกข้อมูลกับฐานข้อมูล PostgreSQL

1.2 การประยุกต์ใช้หลักการ SOLID

- **S - Single Responsibility Principle (SRP):** ทุกคลาสมีหน้าที่เดียว เช่น StudentDiscountStrategy มีหน้าที่แค่คำนวณส่วนลด 10%
- **O - Open/Closed Principle (OCP):** ระบบเปิดรับการขยาย หากในอนาคตต้องการเพิ่ม "ส่วนลด VIP 30%" สามารถสร้างคลาสใหม่มา implements DiscountStrategy ได้เลย โดยไม่ต้องแก้ไขโค้ดเก่า
- **L - Liskov Substitution Principle (LSP):** คลาสลูก (เช่น SeasonalSaleStrategy) สามารถทำงานทดแทนคลาสแม่ (DiscountStrategy) ได้สมบูรณ์โดยไม่ทำให้ระบบพัง
- **I - Interface Segregation Principle (ISP):** Interface DiscountStrategy มีเฉพาะ method ที่จำเป็นต่อการคำนวณราคาเท่านั้น
- **D - Dependency Inversion Principle (DIP):** GameService ไม่พึ่งพาคลาสคอนกรีตโดยตรง แต่พึ่งพาผ่าน Abstraction (Interface) ผ่าน Constructor Injection

1.3 การประยุกต์ใช้ Strategy Pattern

การคำนวณส่วนลดราคาเกมถูกออกแบบด้วย Strategy Pattern เพื่อแก้ปัญหาการใช้ if-else ซ้อนกัน

- **Interface (DiscountStrategy):** กำหนดรูปแบบมาตรฐานในการเช็คเงื่อนไขและคำนวณราคา
- **Concrete Strategies:** คลาส NoDiscountStrategy, StudentDiscountStrategy, และ SeasonalSaleStrategy เก็บสมการคำนวณเฉพาะตัว
- **Context (DiscountContext):** ทำหน้าที่ตรวจสอบและสวิตช์ใช้งาน Strategy ที่ถูกต้องตามประเภทส่วนลดที่รับเข้ามาโดยอัตโนมัติ

1.4 เหตุผลที่ต้องแยก Service Layer ออกจาก Controller และ Repository

การแยก **Service Layer** ออกมาเป็นตัวกลาง ให้ประโยชน์หลัก 2 ประการตามหลักสถาปัตยกรรมซอฟต์แวร์

1. **High Cohesion (ความเกาะเกี่ยวภายในสูง):** แต่ละคลาสโฟกัสเฉพาะหน้าที่หลักของตนเอง Controller จัดการแค่ Request/Response, Repository คุยแค่กับ Database และ Service จัดการแค่การคำนวณและกฎเกณฑ์ทางธุรกิจ ทำให้โค้ดอ่านง่าย
2. **Low Coupling (ความพึ่งพากันระหว่างโมดูลต่ำ):** หากนำ Logic ทุกอย่างไปรวมใน Controller จะทำให้เกิดปัญหา Controller ผูกติดกับ Database การมี Service มาคั่นกลางช่วยลดการพึ่งพา หากต้องการเปลี่ยนกฎธุรกิจ (เช่น เปลี่ยนสมการส่วนลด) ก็แค่แก้ที่ Service โดยไม่กระทบ Controller หรือ Repository ทำให้ระบบยืดหยุ่น

1.5 ลำดับการทำงาน (Execution Flow)

1. **Browser** ส่ง HTTP Request มาที่ Path ของระบบ (เช่น GET /games)
2. GameController รับ Request และเรียกใช้งาน getAllGames() จาก GameService
3. GameService สั่ง GameRepository ให้ดึงข้อมูลจาก PostgreSQL
4. GameService นำข้อมูลที่ได้ มาเรียกใช้ DiscountContext (Strategy Pattern) เพื่อคำนวณราคาสุทธิ (finalPrice)
5. ส่งข้อมูลทั้งหมดกลับไป GameController เพื่อส่งต่อไปยัง **Thymeleaf Template** แปลงผลลัพธ์เป็นหน้า HTML กลับไปที่ Browser

ส่วนที่ 2: Code Implementation & Explanation

โครงสร้าง Code และหน้าที่ของแต่ละ Layer

1. **Entity (Game.java):** เป็นโมเดลข้อมูลที่ผูก (Map) กับตาราง games ใน Database ผ่าน JPA Annotations เช่น @Entity, @Id, @Transient (สำหรับฟิลด์ที่ไม่บันทึกลง DB เช่น finalPrice)
2. **Repository (GameRepository.java):** เป็น Interface ที่สืบทอด JpaRepository ทำให้สามารถใช้งาน CRUD พื้นฐานกับ PostgreSQL ได้ทันทีโดยไม่ต้องเขียน SQL เอง
3. **Strategy Package:** รวมโค้ดสำหรับการคำนวณส่วนลด ประกอบด้วย Interface DiscountStrategy, คลาสย่อยต่างๆ และคลาส DiscountContext ที่ทำหน้าที่เลือกจะใช้ Strategy ตัวไหน
4. **Service (GameService.java):** จัดการ Business Logic ทำหน้าที่เรียกข้อมูลจาก Repository และนำมาผ่านกระบวนการคำนวณราคาจาก Strategy Context ก่อนส่งต่อ
5. **Controller (GameController.java):** จัดการ HTTP Methods (GET, POST) รับข้อมูลจากฟอร์ม ส่งต่อให้ Service ทำงาน และส่งผลลัพธ์กลับไปยัง View (Thymeleaf)

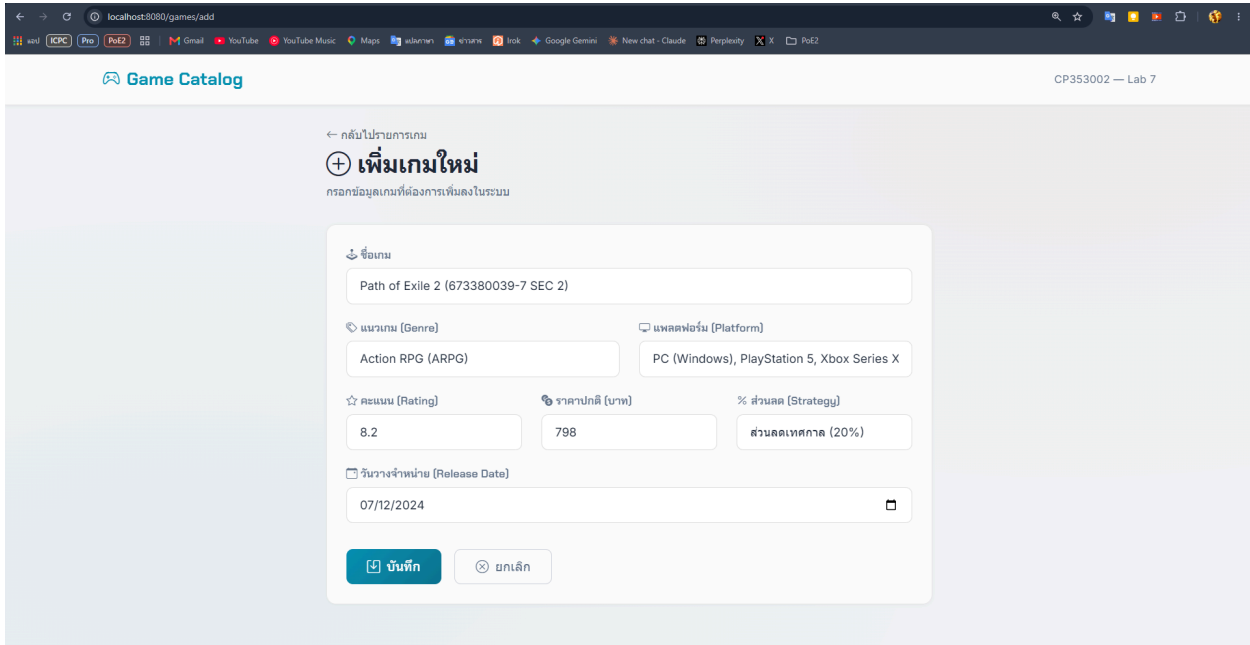
การใช้งาน Dependency Injection (Constructor Injection)

ระบบนี้ปฏิบัติตาม Dependency Inversion Principle (DIP) โดยใช้ Constructor Injection ในทุก Layer แทนการใช้ @Autowired ที่ตัวแปรโดยตรง ซึ่งช่วยให้มั่นใจได้ว่า Dependency ที่จำเป็นต้องถูกส่งเข้ามาเสมอเมื่อมีการสร้าง Object:

- ใน **Controller:** มีการ Inject GameService ผ่าน Constructor
- ใน **Service:** มีการ Inject GameRepository และ DiscountContext ผ่าน Constructor
- ใน **Strategy Context:** มีการ Inject List<DiscountStrategy> ผ่าน Constructor ซึ่ง Spring จะทำการรวบรวมคลาสส่วนลดทั้งหมดที่ใส่ @Component มาใส่ให้โดยอัตโนมัติ

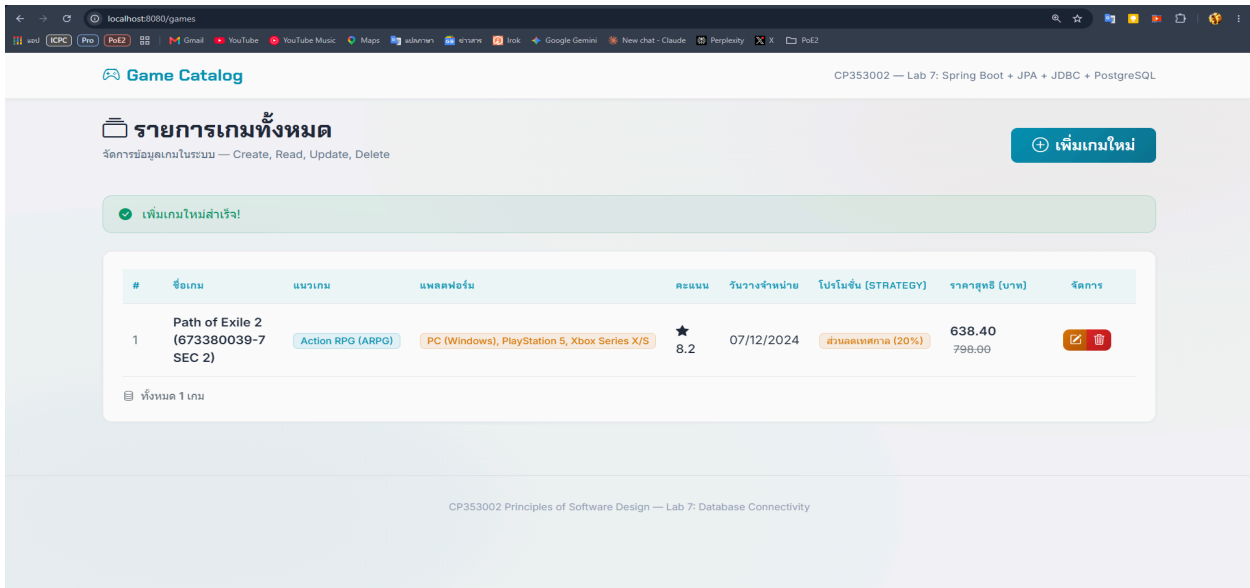
ส่วนที่ 3: Web Application & Database Screenshots

3.1 หน้าจอการเพิ่มเกมใหม่ (Create)



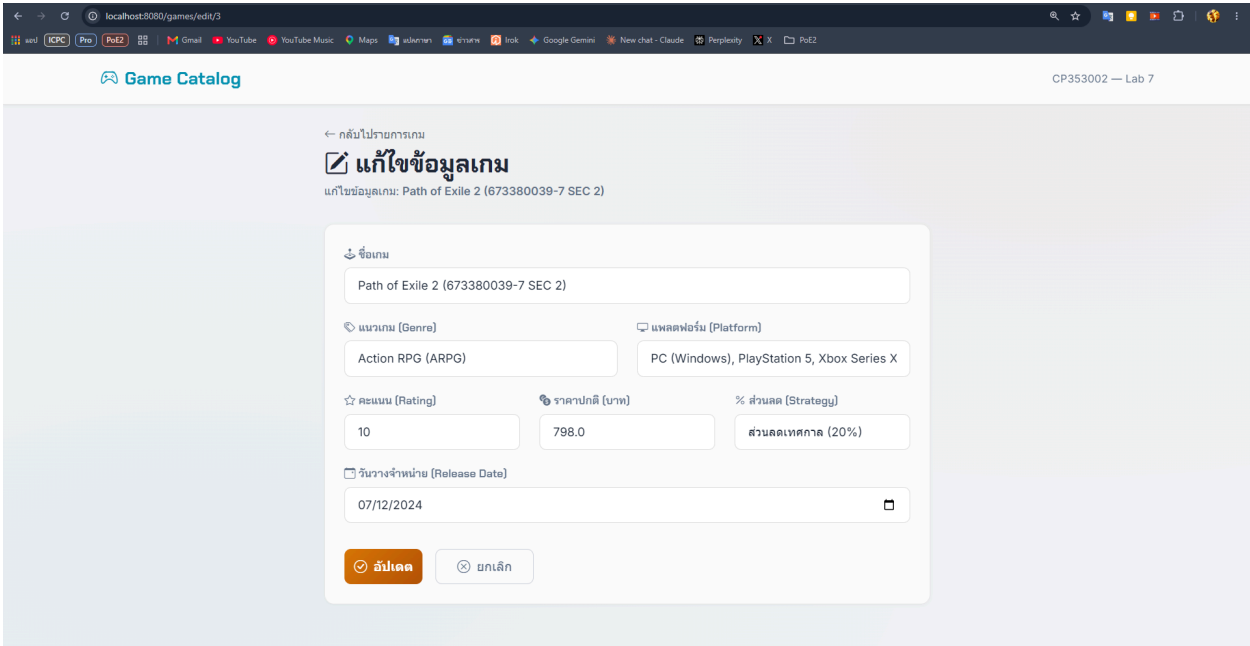
รูปที่ 3.1: หน้าฟอร์มเพิ่มเกมใหม่ (ระบุรหัสนักศึกษา 673380039-7 Section 2 และเลือกส่วนลดนักศึกษา 20%)

3.2 หน้าจอแสดงรายการเกมทั้งหมด (Read)



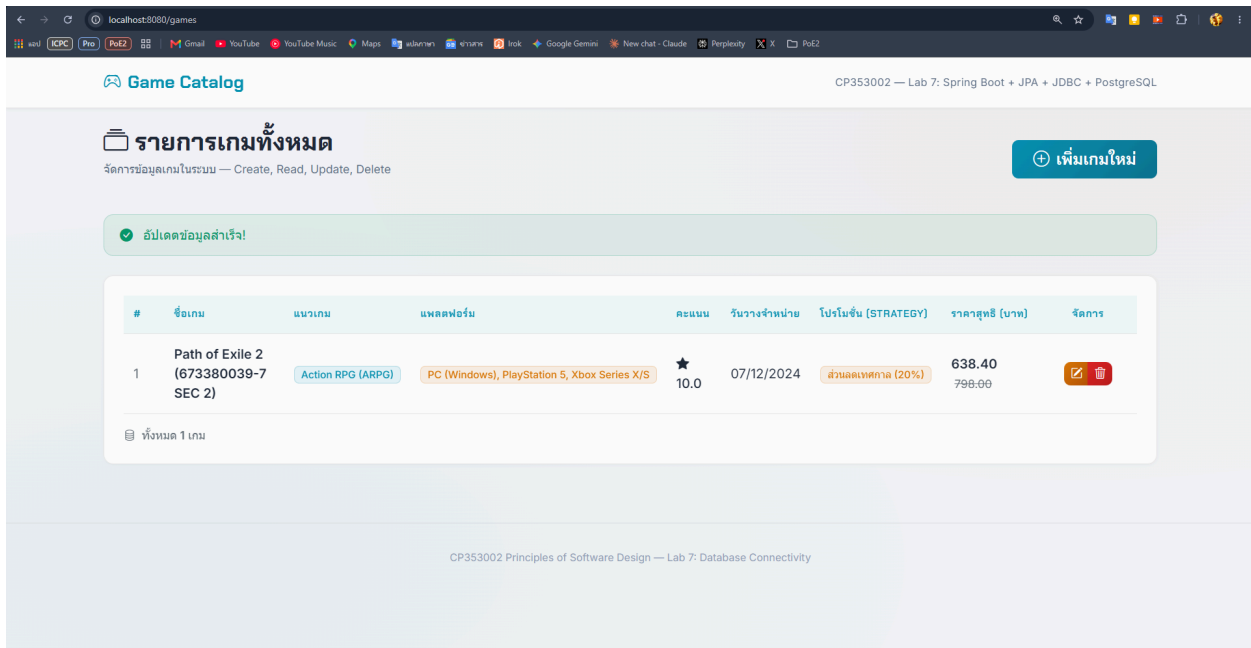
รูปที่ 3.2: หน้าแสดงรายการเกมทั้งหมด พร้อมแสดงราคาสุทธิที่คำนวณผ่าน Strategy Pattern

3.3 หน้าจอแก้ไขข้อมูลเกม (Update)



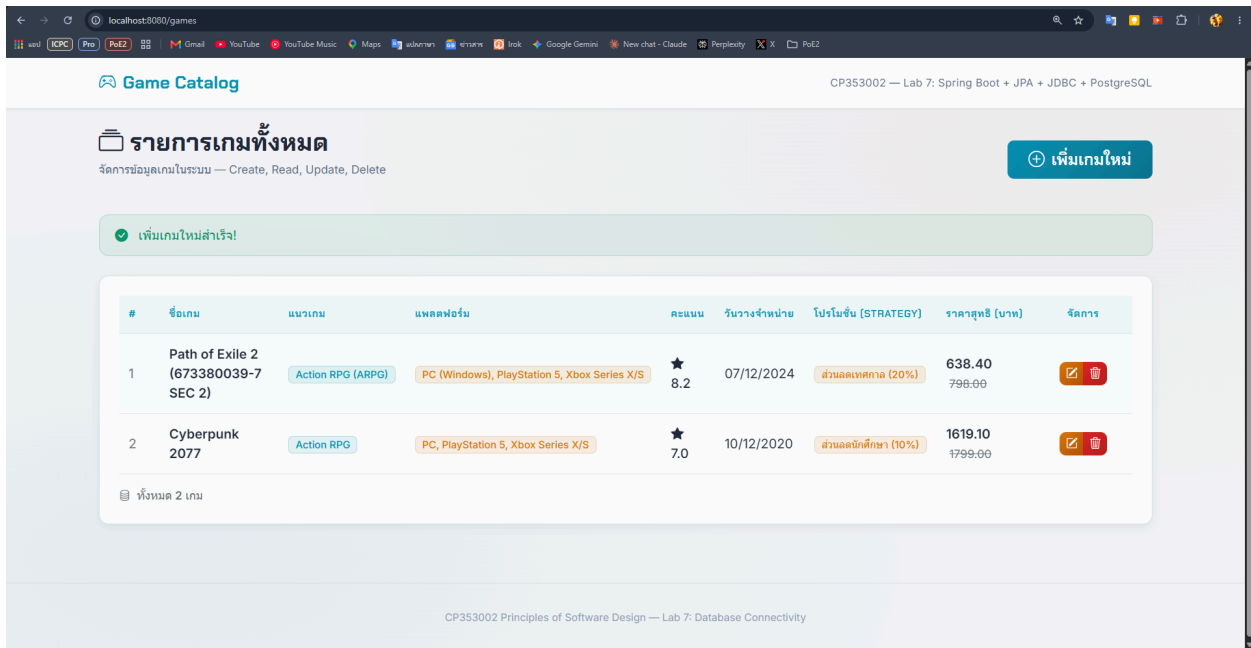
รูปที่ 3.3: หน้าฟอร์มแก้ไขข้อมูลเกม

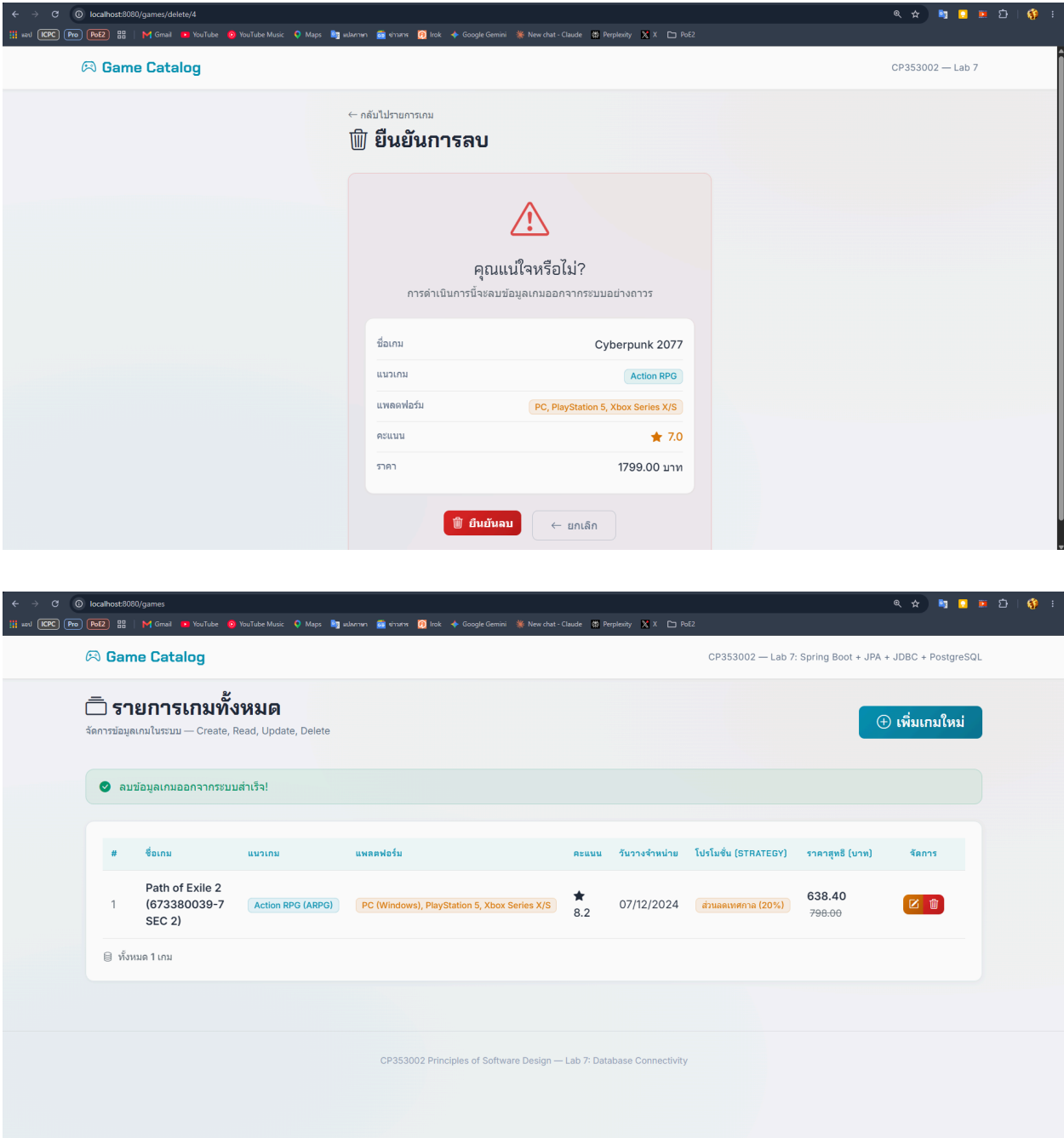
3.4 หน้ารายการหลังแก้ไขสำเร็จ (List Updated)



รูปที่ 3.4: หน้าตารางรายการเกม ที่แสดงผลข้อมูลที่ได้รับการแก้ไขแล้ว

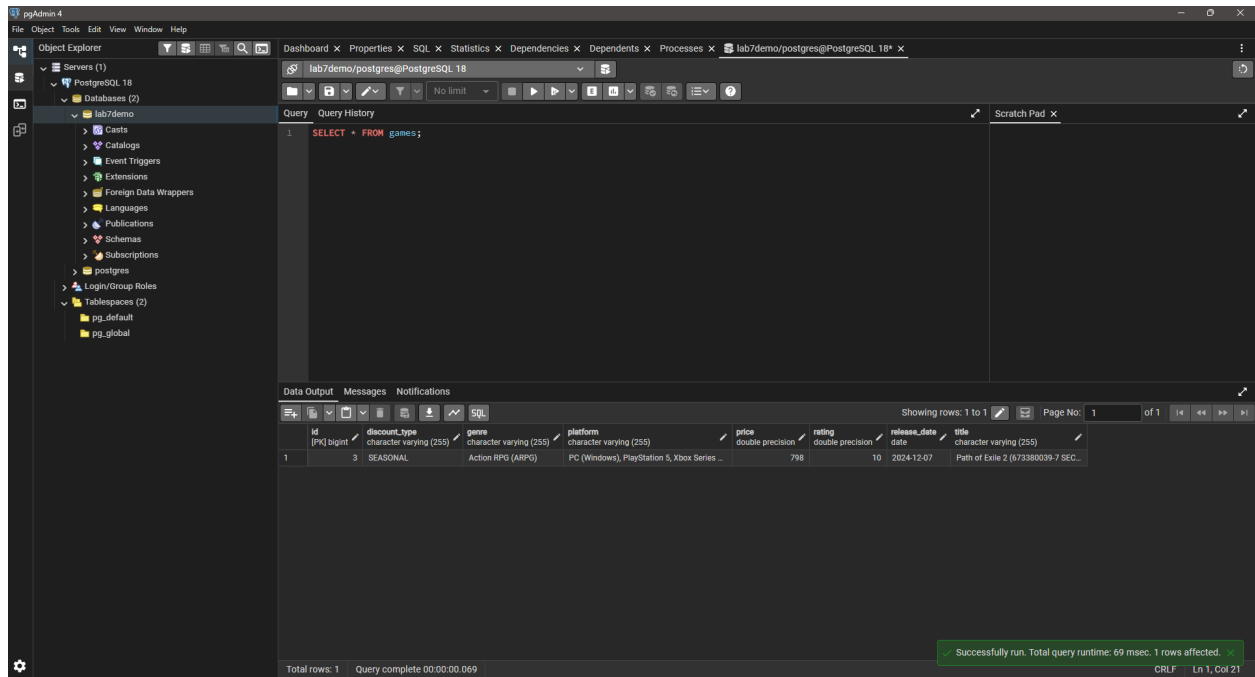
3.5 หน้าจอยืนยันและการลบข้อมูลเกม (Delete)





รูปที่ 3.5: หน้าจอยืนยันการลบเกม และผลลัพธ์หลังทำการลบ

3.6 หน้าจอตรวจสอบข้อมูลใน PostgreSQL Database



รูปที่ 3.6: ข้อมูลตาราง games ที่ถูกจัดเก็บจริงในฐานข้อมูล PostgreSQL